



Structuri de Date și Algoritmi (SDA)

As.Drd.Ing. Bianca Gușiță

AC/TI, AN II

LAB 8 – Tehnici de căutare în tablouri. Funcții HASH



LAB 8 – Tehnici de căutare în tablouri

Tipul de date abstract tablou:

Tabloul reprezintă o secvență de elemente de același tip, numit tip de bază.

- Accesul la elemente se realizează cu ajutorul unui index asociat, care trebuie să aparțină unui tip ordinal finit.
- Indexul precizează poziția unui element în cadrul tabloului.

Notății utilizate:

- TipElement - tipul fiecărui element al tabloului (tipul de bază);
- a - tablou unidimensional;
- i - index;
- e - obiect (valoare) de tip TipElement.

Operații elementare:

- ❖ ScribeTablou(a, i, e) - funcție care scrie valoarea lui e în cel de-al i -lea element al lui a
- ❖ CitesteTablou(a, i) - funcție care returnează valoarea celui de-al i -lea element al lui a



LAB 8 – Tehnici de căutare în tablouri

Implementarea structurii și a operațiilor în C:

```
1  /*Definirea unui tablou */
2  tipElement numeTablou[nrElemente];
3  -----
4  /* Definirea unui tip tablou */
5  typedef tipElement tipTablou[nrElemente];
6  tipTablou numeTablou;
7  -----
8  /* Definirea unui tablou exemple */
9  float tablou[10]; // tablou cu elemente de tip float
10 char sir[10]; // tablou cu elemente de tip char
11 -----
12 /* Definirea unui tip tablou exemplu */
13 typedef float tipTablou[100];
14 tipTablou a, b;
```

LAB 8 – Tehnici de căutare în tablouri



O valoare structurată (o instanță) a unui tip de date TipTablou având componentele **c1, c2..., cn** poate fi inițializată printr-un constructor de tablou și o operație de atribuire:

```
tipElement numeTablou[nrElemente]={expr_const0, expresie_const1,...};
```

```
/* Construcția unui tablou */  
int vect[5]={0,1,33,4,8};  
int mat[2][3]={{11,12,13},{1,2,3}};
```

Tipul tablou este un tip structurat fundamental, deoarece este definit direct în limbaj, având ca suport hardware adresarea indexată.

Una dintre operațiile cele mai frecvent executate asupra tablourilor este căutarea, adică verificarea existenței unui element dat într-un tablou.

Problema căutării consta de fapt în a determina cel mai mic indice **i** pentru care **a[i]=x**, unde **x** este o valoare aparținând lui **TipElement**.

În continuare vom considera ca **TipIndex** este tipul subdomeniu **1..N (N>0)**.

LAB 8 – Tehnici de căutare în tablouri.

Căutarea liniară



Se compară pe rând elementele tabloului cu elementul căutat (x) până când acesta este găsit (se găsește egalitatea $a[i]=x$), fie până când s-a ajuns la sfârșitul tabloului (elementul nu a fost găsit în tablou).

```
int i=0;
for(i=0; (i < N - 1) && (a[i] != x); i++);
if (a[i] == x ) {
    /*elementul x se află în tablou la indicele i*/
} else {
    /*elementul nu a fost găsit în tablou*/
}
```

LAB 8 – Tehnici de căutare în tablouri.

Tehnica fanionului



- Căutarea liniară poate fi îmbunătățită prin tehnica fanionului:
- **Tabloul a se prelungește cu încă un element (fanion) căruia i se asignează valoarea căutată x , apoi se aplică metoda de căutare liniară.**
- **Avantaj: simplificarea condiției de ciclare**, în sensul în care nu mai este nevoie să se verifice dacă indicele nu depășește dimensiunea tabloului, deoarece în tablou există sigur cel puțin un element cu valoarea căutată.

```
int i=0;
a[N]=x;
while(a[i]!=x)
    i++;
if (i == N){
    /*în tablou nu există elementul căutat*/
} else {
    /*elementul a fost găsit la indicele i*/
}
```

LAB 8 – Tehnici de căutare în tablouri.

Căutarea binară (logaritmică)



- **Se poate aplica doar pentru tablouri ordonate (tablouri sortate)!**
- **Principiul ei constă în înjumătățirea repetată a intervalului în care se caută elementul dorit.**
- Această tehnică are **avantajul rapidității**: numărul de comparații necesare este *cel mult* **$\log_2(N)$** .

```
int cautare_binara(tip_element a[], int n, int x)
{
    int stanga, dreapta, mij;
    int gasit;
    stanga = 0; dreapta = n - 1; gasit = 0;
    while ((stanga <= dreapta) && (!gasit))
    {
        /* aici este modificarea */
        mij = (stanga + dreapta) / 2;
        if (a[mij].cheie < x) stanga = mij + 1;
        else if (a[mij].cheie > x) dreapta = mij - 1;
        else if (a[mij].cheie == x) gasit = 1;
    }
    if (gasit) /*avem o coincidență la indexul i*/
        return mij;
    else return -1;
}
```

LAB 8 – Tehnici de căutare în tablouri.

Căutarea prin interpolare



- **Este similară cu căutarea binară**(tabloul trebuie să fie sortat), **dar folosește o altă formulă pentru calculul lui m** , și anume:

$$m := s + (x - a[s]) * (d - s) \text{ div } (a[d] - a[s])$$

- Aceasta conduce la o delimitare mai rapidă a zonei din tablou în care s-ar putea găsi elementul căutat x . Ca principiu, metoda este inspirată după procedeul căutării într-o carte de telefon.

```
int cautare_interpolare(tip_element a[], int n, int x)
{
    int stanga, dreapta, mij;
    int gasit;
    stanga = 0; dreapta = n - 1; gasit = 0;
    while ((stanga <= dreapta) && (!gasit) && (x >= a[stanga].cheie) && (x <= a[dreapta].cheie))
    {
        /* aici este modificarea */
        mij = stanga + ((x - a[stanga].cheie)*(dreapta - stanga)) / (a[dreapta].cheie - a[stanga].cheie);
        if (a[mij].cheie < x) stanga = mij + 1;
        else if (a[mij].cheie > x) dreapta = mij - 1;
        else if (a[mij].cheie == x) gasit = 1;
    }
    if (gasit) /*avem o coincidență la indexul i*/
        return mij;
    else return -1;
}
```

LAB 8 – Tehnici de căutare în tablouri.

Căutarea prin interpolare



- Această metodă este eficientă în cazul în care N este foarte mare și valorile elementelor tabloului au o distribuție uniformă în intervalul $a[1], \dots, a[N]$.
- Numărul de căutări în acest caz este de ordinul $\lg(\lg N)$.**
- Aplicarea căutării prin interpolare necesită ca elementul de căutat x , să se afle în interiorul intervalului $a[1], \dots, a[N]$, altfel apare riscul ca valoarea calculată a lui m să depășească N .

```
int cautare_interpolare(tip_element a[], int n, int x)
{
    int stanga, dreapta, mij;
    int gasit;
    stanga = 0; dreapta = n - 1; gasit = 0;
    while ((stanga <= dreapta) && (!gasit) && (x >= a[stanga].cheie) && (x <= a[dreapta].cheie))
    {
        /* aici este modificarea */
        mij = stanga + ((x - a[stanga].cheie)*(dreapta - stanga)) / (a[dreapta].cheie - a[stanga].cheie);
        if (a[mij].cheie < x) stanga = mij + 1;
        else if (a[mij].cheie > x) dreapta = mij - 1;
        else if (a[mij].cheie == x) gasit = 1;
    }
    if (gasit) /*avem o coincidență la indexul i*/
        return mij;
    else return -1;
}
```

LAB 8 – Tehnici de căutare în tablouri.

Funcții HASH



Funcțiile *hash* sunt algoritmi care transformă o bucată de date (text, fișiere, parole etc.) într-o secvență fixă de caractere, numită *hash*.

- Această secvență de caractere este unică pentru setul de date introdus, astfel încât și o mică schimbare a datelor inițiale va genera un *hash* complet diferit.

Importanța funcțiilor hash

Funcțiile *hash* sunt vitale pentru securitatea cibernetică și pentru gestionarea integrității datelor.

Ele permit autentificarea fără a expune informații sensibile și asigură că datele nu sunt alterate în timpul transmiterii sau stocării.

În plus, ele sunt fundamentul unor tehnologii esențiale din criptografie și stocarea de date, oferind o metodă rapidă și eficientă pentru comparații și verificări de date.

LAB 8 – Tehnici de căutare în tablouri.

Funcții HASH



Cum funcționează o funcție hash?

- O funcție *hash* ia o intrare (de exemplu, un text) și o procesează printr-un algoritm matematic, producând un șir de caractere cu o lungime fixă (de exemplu, 256 biți pentru SHA-256).
- Principalele caracteristici ale unei funcții hash sunt:

Determinismul:
aceeași intrare va
genera
întotdeauna
același hash.

**Sensibilitatea la
modificări:** o
mică modificare
în datele inițiale
schimbă complet
hash-ul.

**Imposibilitatea
inversării:** ideal,
un hash nu poate
fi inversat pentru
a obține datele
originale.

**Rezistența la
coliziuni:** e greu
de găsit două
seturi diferite de
date care să
producă același
hash.

LAB 8 – Tehnici de căutare în tablouri.

Funcții HASH



La ce se utilizează funcțiile hash?

Securizarea parolelor:

În loc să stocheze parolele în clar, sistemele de securitate salvează hash-urile parolelor. Când un utilizator se autentifică, parola introdusă este hash-uită, iar rezultatul este comparat cu hash-ul stocat. Așa, parola originală nu este niciodată salvată în baza de date.

Verificarea integrității datelor:

Funcțiile hash sunt folosite pentru a verifica dacă datele nu au fost modificate. De exemplu, când descarci un fișier, uneori primești și un hash al fișierului. După descărcare, poți calcula hash-ul local și, dacă coincide cu cel furnizat, ai siguranța că fișierul nu a fost alterat.

Blockchain și criptomonede:

Funcțiile hash sunt esențiale în structura blockchain-ului, asigurând integritatea și securitatea datelor tranzacțiilor. De asemenea, ele joacă un rol important în procesul de *minare*, fiind folosite pentru a găsi blocuri valide.

Sisteme de stocare a datelor și baze de date: În tabelele hash, funcțiile hash ajută la indexarea rapidă a datelor pentru accesare eficientă.

LAB 8 – Tehnici de căutare în tablouri.

Funcții HASH



Funcțiile hash sunt de diferite tipuri, în funcție de scopul lor:



Fiecare funcție hash are rolul ei și este selectată în funcție de contextul și necesitățile aplicației.

LAB 8 – Tehnici de căutare în tablouri.

Funcții HASH



1. Funcții hash necriptografice

Aceste funcții sunt rapide și utile pentru aplicații care nu necesită securitate ridicată, cum ar fi stocarea și căutarea datelor într-o tabelă hash.

djb2:

Creată de Daniel J. Bernstein, este simplă și eficientă pentru structurile de date cum ar fi tabelele hash. Este sensibilă la intrările diferite, dar nu oferă securitate criptografică.

FNV-1 și FNV-1a (Fowler-Noll-Vo):

Funcții hash simple și rapide, utilizate frecvent în tabele hash și pentru compararea datelor. FNV-1a este o variantă mai îmbunătățită, care adaugă ordine inversă.

MurmurHash:

Funcție hash foarte rapidă, ideală pentru hashing-ul datelor din structuri mari și baze de date. Este rezistentă la coliziuni și folosită adesea în tabele hash din baze de date și alte structuri mari de date.

CityHash și FarmHash:

Dezvoltate de Google, sunt rapide și optimizate pentru hashing-ul unor șiruri de caractere lungi.

LAB 8 – Tehnici de căutare în tablouri.

Funcții HASH



2. Funcții hash criptografice

Aceste funcții sunt mai complexe și sunt utilizate în criptografie, semnături digitale și securitatea datelor. Ele sunt concepute pentru a fi rezistente la coliziuni și inversare, oferind o securitate ridicată.

MD5 (Message Digest 5):

A fost una dintre cele mai utilizate funcții hash criptografice. Totuși, din cauza coliziunilor descoperite, nu mai este considerată sigură pentru criptografie, dar încă se folosește uneori pentru verificarea integrității datelor.

SHA-1 (Secure Hash Algorithm 1):

Folosit mult timp, dar și el considerat nesigur pentru aplicații critice din cauza vulnerabilităților. Este, de asemenea, utilizat pentru verificarea datelor, dar tot mai rar.

SHA-2 (Secure Hash Algorithm 2):

Familia SHA-2 include algoritmi ca SHA-224, SHA-256, SHA-384 și SHA-512. Acești algoritmi sunt mult mai siguri decât SHA-1 și sunt utilizați în mod activ în criptografie, blockchain și securizarea datelor.

SHA-3 (Secure Hash Algorithm 3):

Este un algoritm hash mai nou, diferit de SHA-2, și oferă un nivel ridicat de securitate. SHA-3 este de obicei folosit în aplicațiile critice unde se cere rezistență sporită la atacuri.

LAB 8 – Tehnici de căutare în tablouri.

Funcții HASH



3. Funcții hash pentru parole și securizarea datelor sensibile

Aceste funcții sunt concepute pentru a rezista atacurilor brute-force și au caracteristici suplimentare de securitate, cum ar fi sărurile (*salts*).

bcrypt:

Este utilizat în principal pentru hashing-ul parolelor. Folosește un algoritm de hashing adaptiv, ceea ce înseamnă că poate fi setat să fie mai lent pentru a preveni atacurile brute-force.

scrypt:

Similar cu bcrypt, dar include rezistență la atacuri prin GPU și ASIC. Este folosit pentru hashing-ul parolelor și în aplicații care necesită o securitate foarte ridicată.

Argon2:

Câștigătorul competiției de hashing pentru parole (Password Hashing Competition). Este considerat unul dintre cele mai sigure și eficiente algoritmi de hashing pentru parole.

LAB 8 – Tehnici de căutare în tablouri.

Funcții HASH



4. Funcții hash dedicate blockchain-urilor și criptomonedelor

Acestea sunt adaptate pentru securitate și eficiență în procesele specifice blockchain-ului și minării.

Keccak (SHA-3):

În unele implementări, cum ar fi Ethereum, se folosește o versiune a SHA-3 numită Keccak, care oferă o combinație de securitate și eficiență.

X11, X13, X15:

Seturi de funcții hash (de exemplu, folosite în criptomonedele Dash), care aplică mai multe funcții hash una după alta, pentru a oferi o rezistență sporită la atacuri.

Ethash:

Funcția hash folosită în Ethereum pentru consensul Proof of Work, optimizată pentru a rezista la atacuri cu ASIC.

LAB 8 – Tehnici de căutare în tablouri.

Funcții HASH



- Un exemplu simplu de funcție hash în limbajul C este o implementare de tip "djb2" (creată de Daniel J. Bernstein).
- Este o funcție hash destul de rapidă și ușor de înțeles, utilizată frecvent în structuri de date precum tabelele hash.

```
1  #include <stdio.h>
2
3  unsigned long hash(const char *str) {
4      unsigned long hash = 5381;
5      int c;
6
7      while ((c = *str++)) {
8          hash = ((hash << 5) + hash) + c; // hash * 33 + c
9      }
10
11     return hash;
12 }
13
14 int main() {
15     const char *text = "Exemplu de text";
16     unsigned long hash_value = hash(text);
17
18     printf("Hash-ul pentru '%s' este: %lu\n", text, hash_value);
19
20     return 0;
21 }
```

LAB 8 – Tehnici de căutare în tablouri.

Funcții HASH – exemplu rezolvat



- CERINȚĂ

- Se dorește crearea unui program în limbajul C care să verifice dacă o listă de nume de utilizatori pentru o aplicație este unică. Fiecare nume de utilizator este introdus de la tastatură, iar programul va folosi o funcție *hash* pentru a stoca și compara numele utilizatorilor introduși. Dacă un utilizator încearcă să adauge un nume deja existent, programul va afișa un mesaj de eroare.

- SOLUȚIE

- Vom folosi o funcție hash pentru a transforma fiecare nume într-o valoare numerică unică și vom folosi o structură de date bazată pe o tabelă hash pentru a verifica rapid dacă un nume a fost deja introdus. Vom alocă un număr maxim de utilizatori pentru a păstra dimensiunea tabelii hash în limite rezonabile.



LAB 8 – Tehnici de căutare în tablouri. Funcții HASH – exemplu rezolvat

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <stdlib.h>
4
5 #define TABLE_SIZE 1000 // Dimensiunea tabelui hash
6 #define MAX_NAME_LENGTH 100
7
8 // Structura pentru un nod din tabela hash
9 typedef struct Node {
10     char name[MAX_NAME_LENGTH];
11     struct Node *next;
12 } Node;
13
14 // Funcție hash de tip djb2 pentru a genera hash-ul numelor
15 unsigned long hash(const char *str) {
16     unsigned long hash = 5381;
17     int c;
18     while ((c = *str++)) {
19         hash = ((hash << 5) + hash) + c;
20     }
21     return hash % TABLE_SIZE;
22 }
23
```

LAB 8 – Tehnici de căutare în tablouri. Funcții HASH – exemplu rezolvat

```
24 // Tabelă hash (array de pointeri la noduri)
25 Node *hash_table[TABLE_SIZE];
26
27 // Funcție pentru a adăuga un nume în tabela hash
28 int add_user(const char *name) {
29     unsigned long index = hash(name);
30
31     // Verificăm dacă există deja un nod la acest index
32     Node *current = hash_table[index];
33     while (current != NULL) {
34         if (strcmp(current->name, name) == 0) {
35             // Numele există deja
36             return 0;
37         }
38         current = current->next;
39     }
40
41     // Numele nu există, adăugăm un nod nou
42     Node *new_node = (Node *)malloc(sizeof(Node));
43     if (!new_node) {
44         printf("Eroare la alocarea memoriei\n");
45         return -1;
46     }
47     strcpy(new_node->name, name);
48     new_node->next = hash_table[index];
49     hash_table[index] = new_node;
50
51     return 1;
52 }
```

LAB 8 – Tehnici de căutare în tablouri. Funcții HASH – exemplu rezolvat

```
54 int main() {
55     char name[MAX_NAME_LENGTH];
56     int n, i;
57
58     printf("Introdu numarul de utilizatori: ");
59     scanf("%d", &n);
60
61     for (i = 0; i < n; i++) {
62         printf("Introdu numele utilizatorului %d: ", i + 1);
63         scanf("%s", name);
64
65         int result = add_user(name);
66         if (result == 0) {
67             printf("Eroare: Numele '%s' exista deja!\n", name);
68         } else if (result == 1) {
69             printf("Numele '%s' a fost adaugat cu succes.\n", name);
70         }
71     }
72
73     // Curățare memorie
74     for (i = 0; i < TABLE_SIZE; i++) {
75         Node *current = hash_table[i];
76         while (current != NULL) {
77             Node *temp = current;
78             current = current->next;
79             free(temp);
80         }
81     }
82
83     return 0;
84 }
```



LAB 8 – Tehnici de căutare în tablouri.

Funcții HASH – exemplu rezolvat

Importanța funcției hash!

Hash-ul face posibilă verificarea rapidă a duplicatelor într-o colecție mare, fără a fi nevoie să parcurgem fiecare element. Acest exemplu arată cum o funcție hash poate asigura unicitatea numelor de utilizator, un aspect important în gestionarea datelor din aplicații reale.

- Explicație a rulării programului:

```
Introdu numarul de utilizatori: 4
Introdu numele utilizatorului 1: Ana
Numele 'Ana' a fost adaugat cu succes.
Introdu numele utilizatorului 2: Bogdan
Numele 'Bogdan' a fost adaugat cu succes.
Introdu numele utilizatorului 3: Ana
Eroare: Numele 'Ana' exista deja!
Introdu numele utilizatorului 4: Cristi
Numele 'Cristi' a fost adaugat cu succes.
```